

Automatic RNA secondary structure prediction with a comparative approach

Fariza Tahi^{1#}

Manolo Gouy²

Mireille Régnier^{3 *}

1- Laboratoire La.M.I. - UMR 8042. CNRS / Université Val-d'Essonne / Genopole.
523, place des Terrasses. 91000 Evry, France.

2- Laboratoire de biometrie et biologie évolutive - UMR CNRS 5558 - Université
C. Bernard, Lyon 1. 43 Bd du 11 Novembre 1918. 69622 Villeurbanne, France.

3- INRIA Rocquencourt. 78153 Le Chesnay, France

#To whom correspondence should be addressed.

Corresponding author : Fariza TAHI

Laboratoire La.M.I. - UMR 8042
CNRS / Université Val-d'Essonne
Genopole
523, place des Terrasses

Abstract

This paper presents an algorithm, DCFold, that automatically predicts the common secondary structure of a set of aligned homologous RNA sequences. It is based on the comparative approach. Helices are searched in one of the sequences, called the “target sequence”, and compared to the helices in the other sequences, called the “test sequences”. Our algorithm searches in the target sequence for palindromes that have a high probability to define helices that are conserved in the test sequences. This selection of significant palindromes is based on criteria that take into account their length and their mutation rate. A recursive search of helices, starting from these likely ones, is implemented using the “divide and conquer” approach. Indeed, as pseudo-knots are not searched by DCFold, a selected palindrome (p, p') makes possible to divide the initial

1 Introduction

Many computational methods for the prediction of RNA secondary structures have been developed. We can distinguish two main approaches. The so-called *thermodynamic approach* relies on thermodynamic estimations of structural stability, in order to determine the secondary structure “of minimal energy”, with respect to some constraints [Gaspin et al., 1995, Sankoff et al., 1983, Winchy et al., 1999]. Two different kinds of algorithms based on energetic models have been developed : the combinatorial algorithms [Papanicolaou et al., 1984, Gouy et al., 1985] and the recursive algorithms [Zuker and Stiegler, 1981]. A detailed description of these methods is given in [Gouy, 1987, Zuker, 1989] and a survey can be found in [Mathews et al., 2000]. This approach is limited by a combinatorial explosion ; hence, it can be used only for small sequences, typically one or two hundred of nucleotides long. Long sequences, such as the ribosomal RNA, cannot be treated by this approach. The *comparative approach* [James et al., 1989] is more appropriate. It is based on a comparison of homologous RNA sequences, which are assumed to have the same secondary structure, i.e. the same helices, called conserved helices. Thanks to this approach, biologists have succeeded to determine, manually, the secondary structures of ribosomal RNA 16S and 23S, the sizes of which are around some thousands of nucleotides [Larsen et al., 1993, Gutell et al., 1985]. Some automatizations of this approach have been proposed, that are based on the dynamic programming. The first algorithm was designed in [Han and Kim, 1993]. It has an $O(m \times n^2 + n^3)$ time complexity, where m is the number of homologous sequences involved and n the maximal size of these sequences. In [Sakakibara et al., 1994], [Eddy and Durbin, 1994] and [Grate, 1995], stochastic context-free grammars (SCFG) have been used. In [Sakakibara et al., 1994] and [Eddy and Durbin, 1994], the time complexity is $O(n^3 m)$ and the space complexity is $O(n^2 m)$.

In most existing algorithms, pseudo-knots are not allowed. They appear in very few cases, almost never; hence, this knowledge does not justify the high complexities that this search generates. It is proved in [Lyngso and Pedersen, 2000] that the general problem of the prediction of a RNA secondary structure containing pseudo-knots is NP-hard for a large class of reasonable models of pseudo-

knots. In [Rivas and S.Eddy, 1999], a dynamic programming algorithm for the prediction of RNA secondary structure is presented, that includes some kinds of pseudo-knots, with an $O(n^6)$ time complexity and an $O(n^3)$ space complexity. In [Lyngso and Pedersen, 2000], the time complexity is $O(n^5)$.

In this paper, we propose an original implementation of the comparative approach, DCFold, which yields an $O(mn)$ space complexity and an $O(mnk)$ time complexity where k is the depth of the secondary structure. The helices are searched recursively, from the more “likely” to the less “likely”, using the “divide and conquer” approach. The likelihood of a helix depends of its *length* and the number of *compensated mutations* it presents. In our algorithm, pseudo-knots are not allowed. Therefore, a selected helix divides the initial sequence into two independent sequences: the internal one and the concatenation of the two external ones. Other helices can be searched independently in these sequences.

Given a set of aligned sequences, the principle of our algorithm is the prediction of the secondary structure of one sequence, called the “*target sequence*”, using informations from the other sequences, called the “*test sequences*”. The search for helices is done in two steps : first, we search for helices in the *target sequence*, then we check their conservation in the *test sequences*.

2 The algorithmic principles of DCFold

2.1 Definitions

Definition 1 *A RNA secondary structure is composed of a set of helices, bulges and internal and external loops. A helix can be composed of smaller helices, separated by bulges or internal loops.*

A helix of a secondary structure is defined by a *palindrome* that represents a particular kind of repetition in the sequence.

Definition 2 *A palindrome is a couple of words (p, p') such that:*

$$\begin{aligned} (i) \quad & |p| = |p'| = m \\ (ii) \quad & p[k] \mathcal{R}_c p'[m - k + 1], \quad \forall k, 1 \leq k \leq m, \end{aligned}$$

where $\mathcal{R}_c$ is the complementary (Watson-Crick) relation between nucleotides : $A\mathcal{R}_cU$, $G\mathcal{R}_cC$.

A palindrome can also be viewed as a succession of base pairs $A - U$ and $G - C$.

Definition 3 A quasi palindrome is a palindrome that contains the pair $G - U$.

Definition 4 An approximate palindrome is a palindrome (or quasi palindrome) where condition (ii) is violated for some k . Such a position is called a mismatching position and we will say that the palindrome presents an error at that position.

The approximate palindromes define helices with bulges and/or internal loops.

In the following sequence S :

AAAGGUCCCGGAUUUUUAUUAUCCGGCCCUUUAAGAAUCGGGAUAGCUAAAAA
 <-.- <----- <-----> -.-> <---+-- --+*-->
 p2 p1 p1' p2' p3 p3'

the couple of words $(p1, p1') = (CCGGAUU, AAUCCGG)$ is an exact palindrome, the couple $(p2, p2') = (AAGG, CUUU)$ is a quasi palindrome and the couple $(p3, p3') = (AGAAUC, GAAUGCU)$ is an approximate palindrome with one bulge and one internal loop.

A RNA sequence contains a very high number of palindromes and not all of them define helices.

Definition 5 A structural palindrome is a palindrome which defines a helix of the secondary structure.

A palindrome is characterized by its position and its length. Namely:

Definition 6 The position of a palindrome (p, p') in a sequence is a couple of integers (x, y) , where x is the position of the first character of p and y the position of the last character of p' in that sequence. We will say that x is the starting position of the palindrome and y its final position. The length of a palindrome is the length of the corresponding helix, that is to say the common length of p and p' .

Definition 7 Two palindromes (p, p') and (q, q') can appear:

- *interleaved*:

p q p' q'
...-->...-->...<--...<--...

- *disjoint* :

p p' q q'
...-->...<--...-->...<--...

- *embedded*

p q q' p'
...-->...-->...<--...<--...

Two interleaved palindromes define a pseudo-knot. Embedded and disjoint palindromes are sayed compatible.

2.2 The Comparative method

The general principle of our algorithm is to search for palindromes in the target sequence, then to check the *conservation* of these palindromes in the test sequences, in order to select the “most significant” ones to define common helices.

Definition 8 Given a set of aligned sequences, a palindrome (p, p') found in the target sequence is said conserved in the test sequences, if a palindrome appears in each test sequence at the same aligned position, possibly with a small shift. The conservation is exact if the palindromes appearing at the same positions are exact or quasi palindromes. The conservation is approximate if these palindromes are approximate palindromes.

When a palindrome (p, p') appearing in a sequence is conserved, the palindromes found in the other sequences are not necessarily composed of the same pairs of bases. Some mutations may occur. They are *compensated* when the pairing of the palindrome bases still remains possible. For example, in the two aligned sequences below :

AGGCUAAAAAUGUUUUAGCCUUU

-----> <-----

AGGAUAGAAAAUGUUCUAUCCUUU

---*---> <---*---

p

p'

four compensated mutations are found, that are located at the positions noted “*”.

A conserved palindrome is not necessarily structural. One of the central problems in our algorithm is to determine a heuristic criterion to select the ones that are structural. Two criteria are combined : the length of the palindrome and the number of compensated mutations.

As a matter of fact, the length criterion is not selective enough. Among the long palindromes in a given RNA sequence, some are structural and others are not. For example, Figure 1 below shows results obtained on the ribosomal RNA 16S containing around 1550 bp. The four columns represent respectively the minimal

2.3 The Divide and Conquer approach

The search for structural palindromes is done recursively, using the “divide and conquer” approach, in order to limit the amount of searching. This approach is made possible by the exclusion of pseudo-knots.

Let S be a sequence where a palindrome (p, p') is found. Then S rewrites as

length of a “significant” palindrome appearing in a RNA or DNA sequence of length n . For a sequence of one thousand nucleotides, this minimal length is typically around 6 nucleotides.

The Compensated mutation criterion

A helix of the common secondary structure of different RNA sequences can be defined by a palindrome that presents *compensated mutations*. One of the most important problems we had in our algorithm was to define the minimal number of compensated mutations that must appear in a selected palindrome. From many observations on sequences with a known secondary structure, it appears that most structural palindromes have at least one compensated mutation per site. Palindromes that have less compensated mutations but are very long are also structural when they are compatible with the first ones. Our mutation criterion is therefore related to the length criterion. We define a *mutation number* that is the number of compensated mutations. We define the *mutation rate* as the ratio $mutation_number/length$. We search for palindromes with a mutation rate greater than or equal to 1. If no palindrome is found, we search for palindromes that satisfy a second global criterion. Namely, the sum of the two parameters *length* and *mutation_number* must be greater than or equal to $2 \times lmin$.

Palindromes can be conserved approximatively. Our algorithm allows in each sequence one error in the palindrome, and this non compensated mutations contributes negatively (-1) to the total ed mutc f t utations . A single uta ion wish

- Search for palindromes that satisfy the length criterion in the target sequence S .
- Select among them the conserved ones (in the test sequences) which satisfy the global criterion.
- Deduce the “valid set of anchoring points”, i.e. the palindromes selected above that are all mutually compatible.
- Iterate the process on sub-sequences of S (deduced from the subdivision of S by the anchoring points) that are long enough to contain palindromes.

4.1 Structural palindrome search

Palindrome search in the target sequence : In a first step, DCFold searches for palindromes in the target sequence that satisfy the minimal length criterion. This is done efficiently by the use of the suffix tree structure [McCreight, 1976, Chen and Seiferas, 1984, Ukkonen, 1993]. This structure is the more appropriate structure to search repetitions in a sequence. Using an $O(n)$ space complexity, it

manages to find all palindromes in the target sequence.

position $(x + d, y - d)$, where d is a possible shift. The length of the palindrome (q, q') must be greater than or equal to $lmin$, so this shift d must be smaller than $l - lmin$.

The procedure is implemented in an easy way. The subsequence $S[x + d]S[x + d + 1]S[x + d + 2] \dots$ is matched against $S[y - d]S[y - d - 1]S[y - d - 2] \dots$ in an inverse and complementary way (including the match $G - U$).

In order to be very selective and ensure a low complexity, we restrict approximate conservations to the case where at most one error appears in each test sequence and the number of exact matches is greater than or equal to $lmin$. Moreover, approximate conservation is not allowed for palindromes of length smaller than 4 base pairs. The errors are weighted negatively (-1 per error) in the total number of mutations, in order to give a better rank to the palindromes without errors.

Example :

GGCGGCAGGC ... GCCUGCCGCC ...	GGCGGCAGGC ... GCCUCCCGCG ...
-----> <-----	-----> <-----
p p'	q q'
GGCGGCAUGC ... GCCUGCCGCG ...	GGCCGCAUGC ... GCCUGCCGCG ...
-----> <-----	--*--> <--*--
GGCGGCAGGG ... GCCUGCCGCG ...	GGCGGGAGGG ... GCCUGCCGCG ...
-----> <-----	----*--> <--*----
GGCGGCUGGC ... GCAUGCCGCG ...	GGCGUCAUGC ... GCAUGCCGCG ...
----> <----	---*----> <----*----
AGCUACAGGC ... GCCAGUAGCG ...	GGCGGCGUGC ... GCACGCCGCG ...
----> <----	-----> <-----

If $lmin$ is less than or equal to 5, the palindrome (p, p') in the target sequence is conserved in the test sequences and selected and the palindrome (q, q') is conserved approximatively with at most one error per test sequence.

Strong variability regions : When strong variations are observed in a region, some conserved palindromes may be absent from some sequences. We have implemented a procedure that treats this problem in the case where the sequences have

different sequence lengths in these regions. Given a region, the conservation of target sequence palindromes is checked only in the test sequences with a length close to the target sequence length.

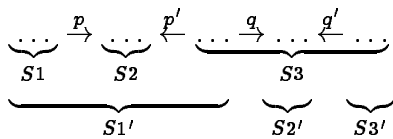
Palindrome compatibility : In order to validate a set of conserved palindromes selected at some step of the recursive search, we have implemented a procedure which deals with non compatible palindromes (if any). When two palindromes are incompatible, we select the one that has the higher selection criterion, i.e. the higher mutation rate. When these mutation rates are equal, we choose the longest palindrome, unless both have the same length: in that case, both palindromes are rejected. We reject as well all the palindromes incompatible with them that either have a smaller mutation rate or have the same mutation rate but a smaller length. One says that the procedure of incompatibility treatment fails if this leads to eliminate all palindromes.

Note that in practice, this procedure is called in a very few cases, as the selection criteria we defined are usually sufficient to select structural palindromes.

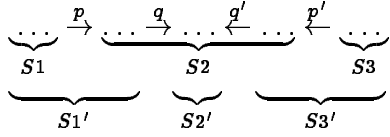
4.2 Recursive calls

The implementation of the recursive calls is not a trivial task. As a matter of fact, when two different palindromes are considered independently, the sub-sequences deduced from the division of the initial sequence can overlap. This is illustrated in the two following schemes, where $S1, S2, S3$ are the sequences deduced from the palindrome (p, p') and $S1', S2', S3'$ are the ones deduced from the palindrome (q, q') :

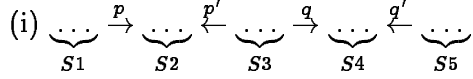
(i) (p, p') and (q, q') are disjoint :



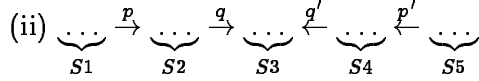
(ii) (q, q') is embedded in (p, p')



The subdivision is performed as follows : first, the palindromes are ordered by the end position. Then, the internal sequences (that can be the concatenation of sub-sequences) are treated sequentially. In the cases (i) and (ii) above, we proceed as follows :

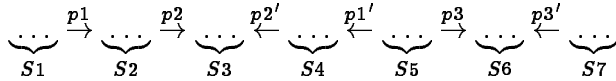


In this case, the sub-sequences are treated in the following order : S2, S4 then S1S3S5 (the sub-sequence resulting from the concatenation of S1, S3 and S5).



In this case, the sub-sequences are treated in the following order : S3, S2S4 then S1S5.

Both cases can occur simultaneously in the same sequence. For example, in the following sequence S :



the sub-sequences are processed in the following order : S3, S2S4, S6 and finally S1S5S7. Their concatenation represents the global sequence without the interleaved structural palindromes.

This recursive process *stops* in two cases.

- the sub-sequence is too small to contain a helix. This is the normal termination case.

More precisely, we consider that the smallest size of a helix is equal to three base pairs, and that the smallest size of a terminal loop is equal to four nucleotides. Therefore, the minimal length of a sequence where we search for palindromes is ten.

- no anchoring points is found.

This case can be due to a *strong variability* of the considered region, i.e. to the presence of an important number of mutations which are not compensated. Alternatively, it can be due to a failure of the incompatibility treatment procedure. This case appears notably when the detected palindromes present zero compensated mutations.

To resume, our search for structural palindromes is done by a recursive procedure described below, *Structural_Palindrome_Search*. It is called with the six following parameters :

L_g : the global list of structural palindromes,

S : the target sequence,

n : the size of the sequence S ,

A : the set of aligned test sequences,

d : the maximal authorized shift in palindrome alignment,

k : the maximal number of authorized gaps in the palindromes.

The procedure is launched with an empty list L_g , the global target sequence S with its length n and the set A of the aligned test sequences.

4.3 Overall Complexity

DCFold has an $O(mnk)$ complexity, where n is the length of the target sequence, m the number of the test sequences and k the depth of the derived secondary structure.

The procedure *Anchoring_Points_Search* has an $O(nm)$ time complexity. Using the suffix tree structure, palindromes in a given sequence of length n can be searched in linear time. To get their positions, more time is needed, it is proportional to their number. Therefore, the total searching complexity depends therefore on the number of palindromes found in the target sequence. Nevertheless, as we search for long palindromes, this number is small. As the anchoring points do not overlap, the sum

```

Procedure Structural_Palindrome_Search( $Lg, S, n, A, d, k$ )
Begin
if  $n \geq 10$ 
  then  $L \leftarrow \text{Anchoring\_Points\_Search}(S, n, A, d, k)$ 
    if  $L \neq \emptyset$  then
       $\{ L_g \leftarrow L_g \cup L;$ 
        1. Sort  $L$  according to the increasing order of the end
           positions of the palindromes
        2. for each palindrome  $P$  in  $L$ 
          begin Let  $S_i$  the internal sequence of  $P$  and  $n_i$  its size
            Structural_Palindrome_Search( $Lg, S_i, n_i, A, d, k$ )
          end
        3. Let  $S_e$  the global sequence  $S$  without the sub-sequences  $S_i$ 
           and  $n_e$  its size
            Structural_Palindrome_Search( $Lg, S_e, n_e, A, d, k$ )
       $\}$ 
End

```

Figure 2: Global recursive procedure of structural palindrome search

of these selected palindromes length, which are greater or equal to $\log_4 n$, cannot exceed the size n of the target sequence. Moreover, checking their conservation in a test sequence and computing the mutation number is achieved in $O(n)$ time.

At each level of the recursive search of structural palindromes, we search for anchoring points in several disjoint sub-sequences of the initial sequence. The total length of these sub-sequences is therefore less than n , which gives a time complexity nearby $O(mn)$. When the number of recursion levels is k , we have a total time complexity of $O(mnk)$.

The space complexity of the algorithm is equal to $O(mn)$. Only one sequence, the target sequence, needs an additional data structure. One may use the suffix tree structure which needs only linear space. The selected palindromes need to be stored,

but their total number cannot exceed n , so the complexity of the data structure is $O(n)$.

5 Results and discussion

The algorithm was tested on transfer RNA and on ribosomal RNA sequences, 16S and 23S.

5.1 Transfer RNA

The RNA transfer is a small molecule, having less than one hundred of nucleotides. It has a secondary structure with a well known structure. We have considered four sequences of tRNA genes of *bacteriophage* and *archeobacteries* : *phage T5*, *Archaeoglobus fulg.*, *Halobacterium cut.*, *Halobacterium hal.*. All are made of approximately seventy nucleotides. The initial alignment was done with *Clustal* algorithm [Thompson et al., 1994].

We searched for conserved palindromes with minimal length 4 and a mutation rate greater than or equal to 1. We found the four palindromes depicted in Figure 3:

<i>beg</i>	<i>end</i>	<i>length</i>	<i>comp_mut</i>	<i>mut_rate</i>
1	71	7	4	1.142857
10	25	4	2	1.000000
27	43	5	4	1.600000
48	64	5	3	1.200000

Figure 3: Structural palindromes in tRNA

All these palindromes are structural. They define the four helices of the transfer RNA secondary structure. Therefore, our algorithm has succeeded to find in a single iteration all the tRNA secondary structure, using only four sequences. Figure 4 below shows the pertinence of our criteria, e.g. the combination of the length and the number of mutations. Among the conserved palindromes with a length greater than or equal to 4, the ones that are not structural have a very low mutation rate. More precisely, their mutation rate ranges from 0.0 to 0.25.

<i>beg</i>	<i>end</i>	<i>length</i>	<i>comp_mut</i>	<i>mut_rate</i>
16	36	4	0	0.250000
50	71	4	0	0.000000
50	61	5	0	0.200000
51	60	4	0	0.250000

Figure 4: Non-structural palindromes in tRNA

5.2 Small and large subunit ribosomal RNA

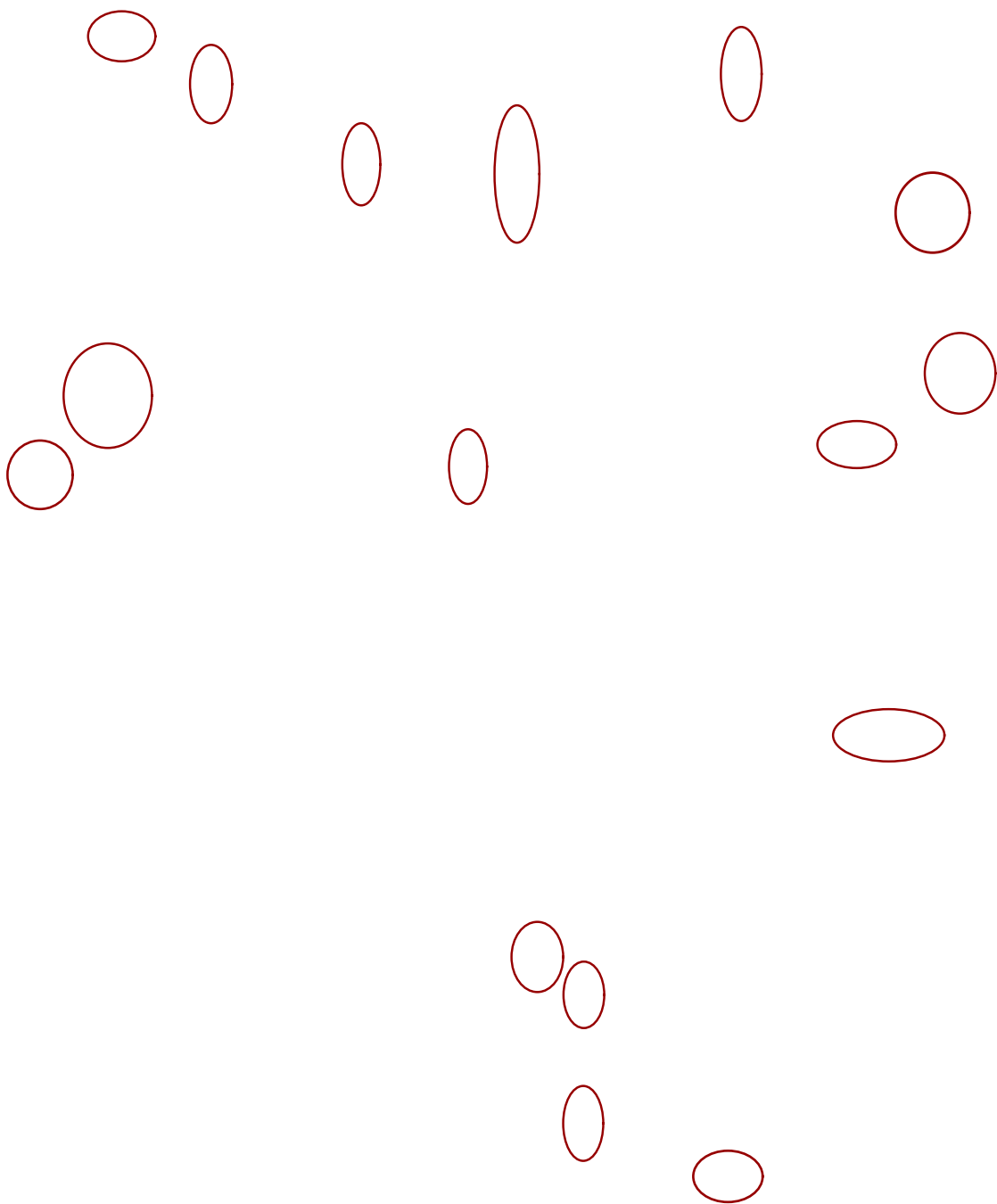
The algorithm DCFold has been tested on ribosomal RNA sequences, 16S and 23S.

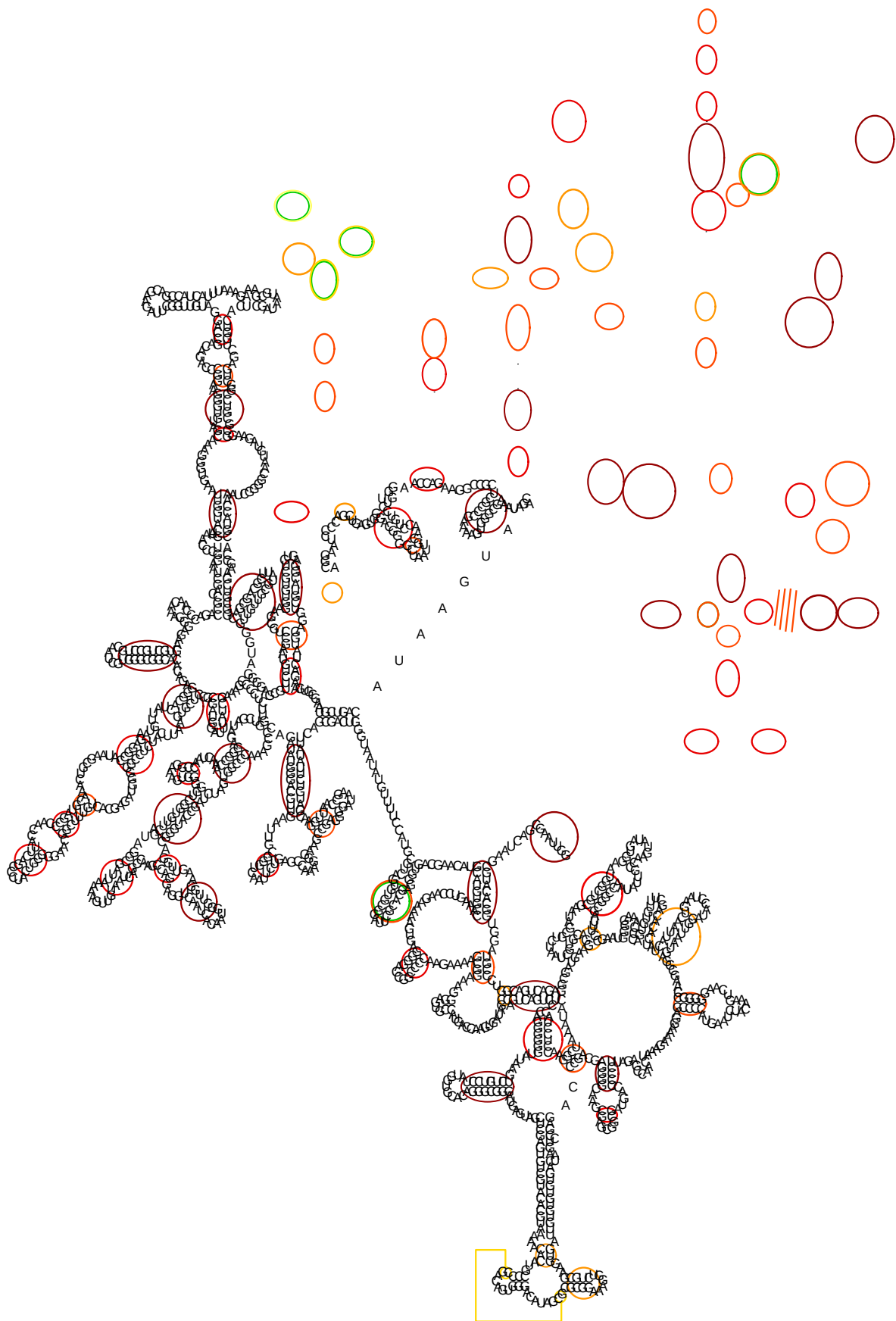
For the 16S ribosomal RNA, we have used seven sequences : Haemophilus influenzae, Neisseria gonorrhoeae, Chlamydia psittaci, Bacillus subtilis, Mycoplasma capricolum, Synechocystis sp and Proteus vulgaris. 83% of the helices have been detected. The procedure of strong variability region treatment allowed to detect 3 of them. The non detected helices are small and have not a big importance for the secondary structure. The obtained results are shown in Figure 5.

For the 23S ribosomal RNA, ten sequences of different species have been used : Campylobacter coli, Clostridium botulinum, Pseudomonas aeruginosa, Chlamydia trachomatis, Ruminobacter amylophilus, Acinetobacter calcoaceticus, Thermotoga maritime Waddlia chondrophila, Anacystis nidulans and Thermus thermophilus. As we can see it on the Figure 6, 81% of the helices have been found.

The number of recursive levels was around 7 in the case of the 16S sequences and 6 in the case of the 23S sequences. The execution time of our algorithm on these two examples is very interesting : five seconds for the rRNA 16S and ten seconds for the rRNA 23S.

Some regions are more difficult to treat than others. It is the case of regions with a *strong variability*, i.e. regions presenting an important number of non compensated mutations, especially deletions and substitutions. It is also the case of regions with a *null variability*, i.e. regions presenting a very low number of mutations.





5.3 Discussion and further work

The results obtained necessarily depend on the set of test sequences used. These sequences must be distant enough from the target sequence to have compensated mutations but close enough to have a minimum of differences in the secondary structure. The choice of these sequences can be done using the phylogenetic tree associated to the set of the sequences, if it exists. To help the user to choose the sequences, we have developed a procedure which compares the test sequences to the target sequence in order to provide some informations : number of substitutions and deletions. Additionally, the number of test sequences must be at least equal to the threshold of relevance, in order to ensure a minimum of compensated mutations (when we suppose that we have one compensated mutation per palindrome per sequence).

The results also depend on the *alignment quality*. To improve its robustness when errors occur in the alignment, we are designing a technique to adjust the alignment, according to the palindromes detected. Moreover, it is interesting to provide a procedure that allows more than one mismatch.

It is worth improving our *threshold on the length* by taking into account the non-uniform letter distribution in DNA or RNA sequences. Given a letter σ from the alphabet $\mathcal{S}$, let p_σ be the probability that the letter σ occurs in a random sequence. Denote $p_{min} = \min_\sigma p_\sigma$ and $p_{max} = \max_\sigma p_\sigma$. Consider the set of words of length m . Their probabilities range from p_{min}^m to p_{max}^m . The larger m is, the wider is the range. Nevertheless, the mass probability tends to concentrate around the average value p_{ave} defined as:

$$p_{ave} = \exp\left(-\sum_{\sigma \in \mathcal{S}} p_\sigma \log p_\sigma\right) . \quad (1)$$

E.g. most of the words have a probability concentrated around p_{ave}^m . A good threshold seems to be given by the equation: $np_{ave}^m = 1$, or, equivalently, $m = \log_{\frac{1}{p_{ave}}} n$.

6 Conclusion

We have presented an algorithm for RNA secondary structure prediction. The algorithm we developed, DCFold, does not use any chemical or thermodynamic constraints. It is based on regularity recognition, considering only some physical constraints connected to the helices and some phylogenetic properties.

This original algorithm, DCFold, is based on the comparative approach. The algorithms proposed in the literature are based on the dynamic programming and consider all possible conserved pairings, before choosing the most “likely” secondary structure. Instead, DCFold searches first adjacent pairings, in other terms the palindromes, in one considered sequence, selects the ones that have some pertinence (using conditions on the length and the number of compensated mutations), and finally checks their conservation in the other sequences. With this approach, the time and space complexities are better than the ones proposed in the literature. Also, we can optimize the searching time for the helices by improving it in a parallel way. Indeed, at each step of the recursive search, the procedure that searches structural palindromes can be launched independently in a parallel way of the disjoint sub-sequences.

The input to the algorithm is a set of aligned homologous sequences. The comparison step of the algorithm depends on this alignment. We are currently working on a new comparison step that would handle errors in the alignment and propose a better alignment, using secondary structure information.

Acknowledgments : We would like to thank Stefan Engelen, an under-graduate student in Bioinformatics at the university of Evry-Val-d’Essonne, who has implemented DCFold in Java. The recent implementation was supported by REMAG Action from INRIA.

References

[Apostolico, 1985] Apostolico, A. (1985). The myriad virtues of suffix trees. In Apostolico, A. and Galil, Z., editors, *Combinatorial Algorithms on Words*, volume 12

of *NATO Advance Science Institute*

- [James et al., 1989] James, B. D., Olsen, G. J., and Pace, N. R. (1989). Phylogenetic comparative analysis of RNA secondary structure. *Methods in Enzymology*, 180:227–239.
- [Larsen et al., 1993] Larsen, N., Olsen, G. J., Maidak, B. L., McCaughey, M. J., Overbeek, R., Macke, T. J., Marsh, T. L., and Woese, C. R. (1993). The ribosomal database project. *NAR*, 21(1):3021–3023.
- [Lyngso and Pedersen, 2000] Lyngso, R. B. and Pedersen, C. N. S. (2000). Pseudoknots in RNA secondary structures. In *RECOMB*, pages 201–209, Tokyo, Japan. ACM 2000.
- [Mathews et al., 2000] Mathews, D. H., Turner, D., and Zuker, M. (2000). RNA secondary structure prediction. In S. Beaucage, D. Bergstrom, G. G. and editors, R. J., editors, *Current Protocols in Nucleic Acid Chemistry*, New York. John Wiley & Sons.
- [McCreight, 1976] McCreight, E. M. (1976). A space-economical suffix tree construction algorithm. *Journal of the Association for Computing Machinery*, 23(2):262–272.
- [Papanicolaou et al., 1984] Papanicolaou, C., Gouy, M., and Ninio, J. (1984). *Nucleic Acids Res.*, 12:31.
- [Rivas and S.Eddy, 1999] Rivas, E. and S.Eddy (1999). A dynamic programming algorithm for RNA structure prediction including pseudoknots. *Journal of Molecular Biology*, 285:2053–2068.
- [Sakakibara et al., 1994] Sakakibara, Y., Brown, M., Hughey, R., Mian, I. S., Sjolander, K., , Underwood, R. C., and Haussler, D. (1994). Stochastic context-free grammars for tRNA modeling. *NAR*, 22:5112–5120.
- [Sankoff et al., 1983] Sankoff, D., Kruskal, J. B., Mainville, S., and Cedergren, R. J. (1983). *Time warps, string edits, and macromolecules : the theory and practice of sequence comparison*. Addison-Wesley Publishing Company.

- [Tahi, 1997] Tahi, F. (1997). *Méthodes Formelles d'Analyse des Séquences de Nucléotides*. PhD thesis, Université de Paris XI (Orsay).
- [Thompson et al., 1994] Thompson, J. D., Higgins, D. G., and Gibson, T. J. (1994). CLUSTAL W: improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position specific gap penalties and weight matrix choice. *Nucleic Acids Research*, 22:4673–4680.
- [Ukkonen, 1993] Ukkonen, E. (1993). On-line construction of suffix-trees. Technical report, University of Helsinki, Department of Computer Science, Finland.
- [Winchy et al., 1999] Winchy, A., Fontana, W., Hofacker, L., and Schuster, P. (1999). Complete suboptimal folding of rna and the stability of these structures. *Biopolymers*, 49:145–165.
- [Zuker, 1989] Zuker, M. (1989). Computer prediction of RNA structure. *Methods Enzymol.*, 180:262–288.
- [Zuker and Stiegler, 1981] Zuker, M. and Stiegler, P. (1981). Optimal computer folding of large RNA sequences using thermodynamics and auxiliary information. *Nucleic Acids Research*, 9:133–148.